

Projeto Clínica Médica — Módulo 03

Turma: LionsDev

Tópicos: Estruturas de Controle, Loops, Arrays, Objetos, Validação de Entrada, Menu Interativo.

1. Visão Geral

Neste projeto você constrói um sistema de agendamento de consultas para uma clínica médica, rodando no terminal com Node.js.

Objetivo de aula: praticar estruturas de controle (`if` / `else`), loops (`while`), arrays e objetos, usando o agendamento de consultas como caso de uso.

O programa deve:

- Oferecer um menu com as ações da clínica;
- Ler os dados da consulta com `prompt` ;
- Guardar cada consulta como um objeto dentro de um array;
- Permitir listar, atualizar e cancelar consultas;
- Validar entradas inválidas (ex.: número de consulta que não existe);
- Permitir encerrar o programa de forma explícita.

2. Por Que Definir Requisitos Antes de Codar?

"Fazer um sistema para uma clínica" pode significar muita coisa: prontuário, receitas, convênios, pagamentos, agenda de vários médicos...

Por isso a gente define antes o que o sistema faz e o que ele não faz. Assim você sabe quando terminou e o que precisa testar.

3. Escopo — Qual Sistema Vamos Criar?

Para este projeto, o escopo inclui:

- Adicionar uma nova consulta (paciente, médico, data e hora)
- Listar todas as consultas
- Atualizar uma consulta existente
- Cancelar uma consulta
- Sair do programa

Cada consulta é um objeto com quatro campos:

```
{ paciente: "Maria", medico: "Dr. Carlos", data: "2026-10-05", hora: "14:30"
}
```

E todas as consultas ficam guardadas em um array:

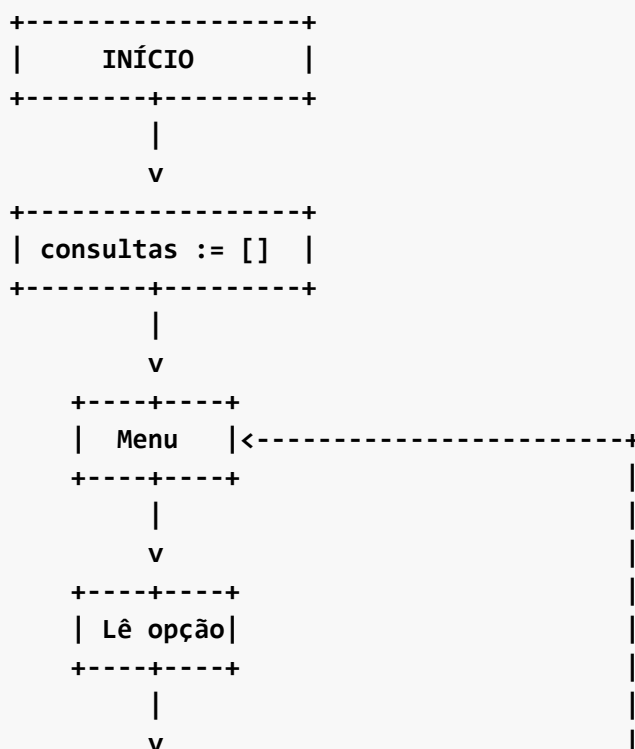
```
let consultas = [];
```

O que fica **fora do escopo** (a menos que você decida estender):

- Cadastro separado de médicos e pacientes;
- Salvar as consultas em arquivo ou banco de dados;
- Interface gráfica (GUI).

4. Fluxo de Execução

1. Criar o array de consultas vazio.
2. Repetir até o usuário pedir para sair:
 - Mostrar o menu no terminal.
 - Ler a opção escolhida (**prompt**).
 - Conforme a opção: adicionar, listar, atualizar ou cancelar uma consulta; ou encerrar.
 - Voltar ao menu (continua o loop).
3. Mensagem de encerramento.



```

+----+----+ opção 0
| if/else +-----> FIM
+----+----+
      |
+-----+-----+
|         |         |         |
v         v         v         v
Adicionar  Listar  Atualizar  Cancelar
|         |         |         |
+-----+-----+
      |
+-----+
      (volta ao Menu)

```

- **Menu:** as opções numéricas (0..4) são tratadas com `if / else if`.
- **Adicionar:** lê os quatro campos, monta o objeto e usa `push` para colocar no array.
- **Listar:** percorre o array e mostra cada consulta com um número na frente (1, 2, 3...).
- **Atualizar e Cancelar:** o usuário escolhe a consulta pelo número mostrado na listagem.
- **Persistência:** as consultas ficam só "em memória" (no array), enquanto o programa estiver rodando. Fechou o programa, os dados somem.

5. Número da Consulta x Posição no Array

Na listagem o usuário vê as consultas começando em **1**, mas o array começa na posição **0**. Por isso, ao atualizar ou cancelar, subtraia 1 do número digitado:

```

Listagem:  1. Maria - Dr. Carlos - 2026-10-05 - 14:30  -> consultas[0]
           2. João  - Dra. Ana   - 2026-10-06 - 09:00  -> consultas[1]

```

```

let num = Number(prompt("Digite o número da consulta: "));
let posicao = num - 1;

```

Para remover uma consulta do array, use o `splice`:

```

consultas.splice(posicao, 1); // remove 1 item a partir da posição

```

6. Validação de Entrada

- **Robustez:** evita apagar ou alterar a consulta errada.
- **Experiência do usuário:** mensagens claras ("consulta não encontrada") em vez de travar.
- **Manutenção:** quando algo dá errado, fica mais fácil saber se foi a entrada.

O número da consulta chega como string pelo `prompt`. O padrão é converter para número e depois validar: primeiro se é um número, depois se existe uma consulta naquela posição.

```
let num = Number(prompt("Digite o número da consulta: "));
let posicao = num - 1;

if (isNaN(num) || posicao < 0 || posicao >= consultas.length) {
  console.log("Consulta não encontrada.");
} else {
  // ... atualiza ou cancela a consulta ...
}
```

7. Escopo Mínimo Obrigatório

CrITÉRIOS para considerar o projeto completo:

- O programa exibe um menu numérico e aceita apenas opções válidas ou informa erro para opção inválida.
- O laço principal só termina quando o usuário escolhe sair.
- Cada consulta é guardada como um objeto com `paciente`, `medico`, `data` e `hora`.
- A listagem mostra todas as consultas numeradas a partir de 1.
- Se não houver consultas, a listagem avisa "Nenhuma consulta agendada." em vez de não mostrar nada.
- Atualizar e cancelar só funcionam com um número de consulta que existe; caso contrário, o sistema avisa e não altera o array.
- Depois de cancelar, a consulta não aparece mais na listagem.

8. Indo Além (Opcional)

O escopo mínimo da seção 7 é obrigatório e precisa funcionar antes de qualquer extra. A partir daí, quem quiser pode acrescentar novas opções ao menu, seguindo o mesmo padrão do projeto: validação de entrada e caso tratado no `if / else`.

Exemplos de extras:

- **Campos obrigatórios:** não aceitar paciente ou médico em branco.
- **Horário ocupado:** impedir duas consultas com o mesmo médico, na mesma data e hora.
- **Buscar por paciente:** listar só as consultas de um paciente digitado.
- **Consultas do dia:** listar só as consultas de uma data escolhida.
- **Contador:** mostrar quantas consultas cada médico tem.
- Outras ideias por conta do aluno: especialidade do médico, telefone do paciente, status da consulta (agendada, realizada, cancelada), etc.

Essas extensões são opcionais e não substituem o escopo mínimo obrigatório.

Próximo passo no curso (código modular):

`modulo06/lista_de_exercicios/agendamento_consultas/` (o mesmo sistema dividido em vários arquivos, com médicos e pacientes cadastrados)

LionsDev • Professor Nicolas Cardoso Motta

Projeto Clínica Médica — Módulo 03